PROJECT WALKTHROUGH

Rice Leaf Disease Detection

From a small labeled image set to model comparison and a Streamlit demo

The project in one minute

This project trains image classifiers to sort rice-leaf photos into three disease categories. The notebook checks the dataset, creates a leakage-aware split, explores augmentation, trains two small convolutional neural networks (CNNs) and an ImageNet-pretrained MobileNetV2, then evaluates each model on a held-out test set. A Streamlit interface lets a user upload an image and inspect the trained model's class scores.

Important context. The archive contains only 119 images and the test set contains 24. This is a learning prototype, not a field-validated crop diagnostic. Model scores on this tiny test set are unstable and must not be presented as production performance.

1. Dataset and labels

The project brief says there are 120 JPEG images with 40 examples in each of three classes. The actual downloaded archive uses an outer ZIP containing one ZIP per category. After safely extracting those inner archives, the notebook finds 40 Bacterial leaf blight images, 40 Brown spot images, and 39 Leaf smut images. That is 119 images. The brief mentions leaf blast in its introductory problem statement, but leaf blast is not one of the archive's listed folder labels. The notebook takes the labels from the folders rather than fabricating a leaf-blast category.

Observed class	Images
Bacterial leaf blight	40
Brown spot	40
Leaf smut	39

Exploratory analysis

The notebook checks that files decode, counts images by class, summarizes original width and height, reports mean RGB channel values, displays sample leaves, and plots the class distribution. OpenCV loads the pixels as BGR by default, so the notebook converts them to RGB before plotting or feeding them to the model. EDA is useful for spotting count discrepancies, varied image sizes, and obvious input problems; it does not prove that the image collection represents real farms.

2. Preprocessing and data split

Images are resized to 96 x 96 pixels for the low-resource run, so every model receives the same input dimensions. The original images are split before any augmentation, using stratification to keep category representation in each subset.

Partition	Images	Purpose
Training	71	Fit model parameters using labeled examples
Validation	24	Monitor model learning during training
Test	24	Final comparison on held-out images

The test set is not used for fitting the model. Keeping it untouched reduces evaluation leakage. It does not eliminate every risk: related or near-duplicate images may already exist in a source dataset, and the small validation/test sets make all estimates noisy.

3. How the image models work

Convolutional neural network (CNN)

A CNN applies learned filters across an image. Earlier layers can respond to local features such as edges or color changes; later layers combine features into patterns useful for classification. This notebook's compact CNN uses convolution and pooling layers, global average pooling, dropout, and a final three-class softmax layer. It is small enough to train on a CPU, but has to learn its visual features from only 71 training images.

Data augmentation

The augmented models apply modest random flips, rotations, zoom, and contrast changes to training examples during training. This can help a model tolerate small changes in viewpoint or lighting. Augmentation is intentionally not used on validation or test images. It cannot create new independent plants, disease examples, or expert labels; overly strong or biologically unrealistic transformations can also change useful visual cues.

Transfer learning with MobileNetV2

MobileNetV2 is a pretrained image model. Transfer learning reuses its ImageNet-trained visual feature extractor instead of learning all visual features from scratch using this tiny dataset. In this project, the pretrained base is frozen and a small new classification head is trained for the three rice-leaf categories. This is a plausible approach when labeled data is scarce, but pretraining cannot correct mislabeled, unrepresentative, or insufficient rice-leaf examples.

4. Results from the executed run

The executed notebook used TensorFlow 2.11 on Python 3.9, with CPU-only execution, 96 x 96 inputs, batch size 4, and 3 epochs. The table below reproduces the saved values from outputs/model_comparison.csv. Macro metrics compute the score for each class and average the three classes equally.

Model	Accuracy	Macro P	Macro R	Macro-F1
MobileNetV2 - transfer + augmentation	79.2%	80.3%	79.2%	78.9%
CNN - no augmentation	33.3%	11.1%	33.3%	16.7%

Model	Accuracy	Macro P	Macro R	Macro-F1
CNN - augmentation	33.3%	11.1%	33.3%	16.7%

How to interpret these numbers

MobileNetV2 with augmentation had the strongest measured result in this split: 79.2% accuracy and 78.9% macro-F1. Each simple CNN had 33.3% accuracy and 16.7% macro-F1. This is evidence only about these specific runs and this split. With 24 test images, one changed prediction moves accuracy by about 4.2 percentage points; each class has only 8 test examples. Repeatedly tuning choices against this test set would also make it less independent. Treat MobileNetV2 as the leading candidate for a larger evaluation, not as a proven production model.

The notebook also writes a per-class classification report, confusion matrices, training curves, image-analysis CSV, model files, and the model-comparison CSV to outputs/. The Streamlit app shows the MobileNetV2 output scores for an uploaded image. These scores are not calibrated probabilities and are not a diagnosis.

5. Streamlit user interface

The app loads the trained .keras model and class names from the notebook-generated outputs. A user chooses a JPEG or PNG image. The interface checks the size and image format, decodes the photo with OpenCV, converts it from BGR to RGB, resizes it to the model's expected 96 x 96 input, and asks TensorFlow for predictions. It displays the top-scoring class and a bar chart of all three model scores. The saved model contains MobileNetV2 preprocessing internally, so the app sends the resized RGB pixel values without duplicating that transform.

Run it from the project folder with the Python environment that can load the saved model: `python -m streamlit run app.py`. The local address used for this project is http://localhost:8501. The app is a demonstration interface; human expertise is still needed for crop-health decisions.

6. Challenges and next steps

Challenge	What was done / what remains
Very small dataset	Use transfer learning and show uncertainty; collect many more independent leaves.
One image missing vs. brief	Report observed 119-image count and per-class counts transparently.
Conflicting label wording	Use archive folder names; verify class definitions with a domain expert.
Evaluation leakage	Split original files first and augment training only; group related captures in future splits.
Possible overfitting	Use validation data and dropout; validation is itself small and limited.
Field variation	Current images may not cover devices, farms, backgrounds, lighting, or severity.

Recommended next steps

• Collect substantially more images across farms, cameras, lighting conditions, growth stages, and disease severity.

• Have a plant-disease expert verify labels and agree on disease definitions.

• Keep an external test set grouped by plant, farm, or capture session; do not tune on it.

• Repeat experiments across several grouped splits and report uncertainty, per-class errors, and calibration.

• Require human review before decisions; do not use this educational model as the sole basis for treatment.

Project files

RiceLeaf_Disease_Detection.ipynb - analysis, training, evaluation, and project report. app.py - Streamlit inference UI. outputs/model_comparison.csv - measured comparison table. README.md - installation, run instructions, and concise project summary.

Prepared from the executed project artifacts. Metrics describe one run only and may vary after retraining.